BDA UT-2 QB

2 Mrks

1) Compare MongoDB with RDBMS

Feature	MongoDB	RDBMS
Type of Database	NoSQL database	Relational database
Data Storage Format	Stores data as **documents (JSON/BSON)**	Stores data in **tables (rows and columns)**
Schema	**Schema-less** (flexible structure)	**Fixed schema** (predefined structure)
Data Relationships	Uses **embedded documents or references**	Uses **foreign keys and joins**
Scalability	Designed for **horizontal scaling** (sharding)	Usually uses **vertical scaling**
Query Language	Uses **MongoDB Query Language (MQL)**	Uses **SQL (Structured Query Language)**
Transactions	Limited transaction support (multi-document supported in newer versions)	Strong **ACID transaction support**
Performance	Faster for **large unstructured or semi-structured data**	Efficient for **structured data and complex queries**
Examples	MongoDB, CouchDB	MySQL, Oracle, PostgreSQL, SQL Server
Use Cases	Big data, real-time apps, content management	Banking systems, enterprise systems, traditional applications

2) Define BSON & its purpose in MongoDB.

Definition:
BSON (Binary JSON) is a binary-encoded format used by **MongoDB** to store and exchange data. It is similar to JSON but is stored in a **binary form**, which makes data processing faster and more efficient.

Purpose of BSON in MongoDB:

1. **Efficient Storage:** BSON stores data in binary format, which reduces parsing time and improves performance.
2. **Faster Data Processing:** Binary encoding allows MongoDB to read and write data quickly.
3. **Supports More Data Types:** BSON supports additional data types such as **Date, ObjectId, Binary data, and Timestamp** that are not available in standard JSON.
4. **Better Indexing:** BSON structure helps MongoDB efficiently create and manage indexes.
5. **Data Traversal:** BSON includes length information, allowing MongoDB to quickly locate and access fields in a document.

3) Draw the PIG Architecture

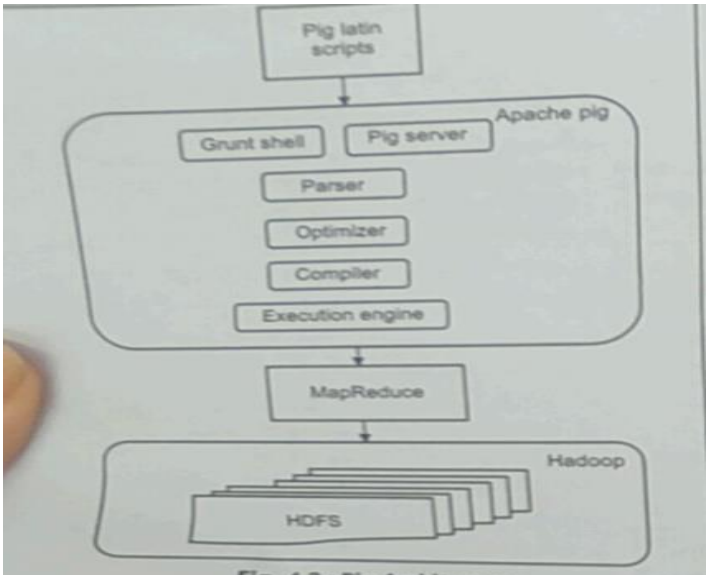


Fig. 4.2 : Pig Architecture

4) Stepwise workflow of PIG Architecture.

Extracted text from the images:

Step-by-Step Workflow of Pig Architecture:

Step 1 : User writes a Pig Latin script using Grunt shell or script file.

Step 2 : Parser checks syntax and creates a Logical Plan.

Step 3 : Optimizer improves the Logical Plan for efficient execution.

Step 4 : Compiler converts it into MapReduce jobs.

Step 5 : Execution Engine submits the jobs to Hadoop.

Step 6 : MapReduce processes the data stored in HDFS.

Step 7 : Results are written back to HDFS or displayed to the user.

5) Char Explain characteristics of HIVE

1. **SQL-Like Language (HiveQL) :** Provides a familiar SQL syntax for querying and managing data.
2. **Schema-on-Read :** The schema is applied only when the data is read, allowing flexibility in data storage.
3. **Batch Processing System :** Designed for long-running queries rather than real-time processing.
4. **Integration with Hadoop :** Works seamlessly with HDFS, YARN, and other Hadoop ecosystem tools.
5. **Supports Various File Formats :** Compatible with Text, ORC, Parquet, and Avro file types.
6. **Metadata Management :** Stores table and partition information in a centralized Metastore.
7. **Extensibility :** Supports user-defined functions (UDFs) to extend its capabilities.

6) Limitations of Hive.

1. **Not Suitable for Real-Time Processing:** Hive queries take time since they are converted into MapReduce jobs.
2. **No Row-Level Updates or Deletes:** Hive works best for read-heavy workloads, not frequent data changes.
3. **High Query Latency:** Response time is slower compared to traditional RDBMS.
4. **Limited Transaction Support:** ACID operations are partially supported.
5. **Dependency on Hadoop:** Hive cannot function without the Hadoop ecosystem.

7) What is spark streaming 2) DStream 3) Write - Ahead Logging (WAL) 4) Checkpointing in spark streaming

Apache Spark Streaming is an extension of the core Spark API that enables scalable, high-throughput, and fault-tolerant stream processing of live data streams. The

DStream: The basic abstraction in Spark Streaming, representing a continuous stream of data as a series of RDDs.

- **Checkpointing:** Stores metadata and data for recovery.
- **Write-Ahead Logs (WAL):** Ensures no data is lost in receivers.

8) Explain the architecture of Spark Streaming with a neat diagram.

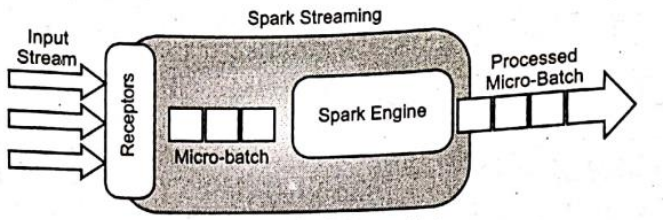


Fig. 5.7 : Spark Streaming

DStream: The basic abstraction in Spark Streaming, representing a continuous stream of data as a series of RDDs.

Receiver: Special objects that capture data from input sources (like Kafka, Flume, or TCP sockets) and store it as RDDs.

Streaming Context: The entry point for Spark Streaming applications, responsible for receiving and processing data streams.

Output Operations: Actions that push processed data to external systems like databases or dashboards.

9) Explain the role of the catalyst optimizer & Tungsten engine in Structured streaming

1. Catalyst Optimizer

Catalyst Optimizer is the **query optimization framework in Apache Spark SQL**. It improves the performance of queries used in Structured Streaming.

Role:

- **Query Analysis:** Converts SQL queries or DataFrame operations into a logical plan.
- **Optimization:** Applies rule-based and cost-based optimizations to improve execution efficiency.
- **Logical to Physical Plan Conversion:** Transforms optimized logical plans into physical execution plans.
- **Efficient Query Execution:** Reduces unnecessary operations such as redundant filtering or scanning.

Result: Faster and more efficient processing of streaming queries.

---

2. Tungsten Engine

Tungsten is the **physical execution engine of Spark** designed to improve CPU and memory efficiency.

Role:

- **Memory Management:** Uses optimized memory management to reduce garbage collection overhead.
- **Binary Processing:** Processes data in a binary format for faster computation.
- **Code Generation:** Uses whole-stage code generation to produce optimized bytecode.
- **Better CPU Utilization:** Improves execution speed by using low-level optimizations.

10) Write short note on : a) Catalyst - Optimizer b) Adaptive Query Execution (AQE)

- **Catalyst Optimizer:** The Catalyst Optimizer is a core component of Spark SQL that applies advanced techniques like cost-based optimization and code generation to dramatically speed up query execution.
- **Adaptive Query Execution (AQE):** AQE dynamically adjusts query plans based on runtime statistics, optimizing joins, partition sizes, and more, leading to improved performance and resource utilization.

ii) Describe the ETL (Extract-Transform-Load) process using Spark using short example.

Steps in Spark ETL

Extract

Data is ingested from various sources such as files (CSV, JSON, Parquet), databases, or streaming platforms.

Spark supports multiple formats and can read data into DataFrames or RDDs for further processing.

Transform

The extracted data is cleaned, validated, and transformed to meet business requirements.

Common transformations include filtering, joining, aggregating, and enriching data.

Spark SQL and DataFrame APIs facilitate these operations with high-level functions and SQL queries.

Load

The transformed data is written to target systems like databases, data warehouses, or file systems.

Spark supports writing data in various formats (CSV, JSON, Parquet, Delta Lake) and can handle both batch and streaming loads.

Short Example

A company collects **sales data from CSV files**. Spark extracts the data, transforms it by **removing duplicate records and calculating total sales**, and then loads the cleaned data into a **data warehouse for reporting**.

4Mks.

i) Explain Pig Latin Data Model with architecture & examples for Tuple, Bag & Map

The Pig Latin Data Model defines how data is represented, stored, and manipulated within Apache Pig. Since Pig is designed to handle structured, semi-structured, and unstructured data, it uses a flexible and nested data model that allows complex relationships between data elements.

Pig's data model supports hierarchical and nested structures

Data Model Architecture (Hierarchy)

The data in Pig is organized into a specific hierarchy:

1. **Field:** The smallest unit of data (e.g., a single string or integer).
2. **Tuple:** An ordered set of fields (similar to a row in a table).
3. **Bag:** A collection of tuples (similar to a table, but unordered and allows duplicates).
4. **Relation:** A bag that represents the entire dataset being processed.

1. Tuple

A Tuple is an ordered set of fields. Each field can be of any data type (scalar or complex). Tuples are enclosed in parentheses ().

Example: ('John', 28)

2. Bag

A Bag is an unordered collection of tuples. It is essentially a "list" of records. Bags are enclosed in curly braces {}. Because they are unordered, you cannot access elements by an index.

- Example: A bag of students in a classroom:

{('Alice', 21), ('Bob', 22), ('Charlie', 20)}

3. Map

A Map is a set of key-value pairs.

- The key must be a chararray (string) and must be unique.
- The value can be of any data type.
- Maps are enclosed in square brackets [], and keys are separated from values by a hash #.
- Example: ['brand'#'Dell', 'RAM'#'16GB', 'Storage'#'512GB']

2) Compare Apach Pig with SQL, Mapreduce & Hive on Parameters / Aspects
1). Languge Type 2). Nature of Processing

FOR EDUCATIONAL USE

Parameter / Aspect	Apache Pig	SQL	MapReduce	Apache Hive
1. Language Type	Pig Latin scripting	Declarative query language	Java programming model	SQL-like (HiveQL)
2. Nature of Processing	Procedural	Declarative	Procedural	Declarative
3. Primary Purpose	Data analysis, ETL	Database queries	Large data processing	Data warehousing
4. Level of Abstraction	High-level	High-level	Low-level	High-level
5. Performance	Faster than MapReduce	Fast for structured data	Slower due to coding	Moderate
6. Execution Model	Runs on MapReduce	Runs on DBMS	Map and Reduce tasks	Converts to MapReduce
7. Use Cases	ETL, log analysis	Transaction queries	Big data processing	Analytics, reporting
8. Output Type	Files or datasets	Table results	Key-value output	Table results
9. Performance Optimization	Automatic optimization	DB query optimization	Manual optimization	Query optimization
10. Error Handling	Easy debugging	DB handles errors	Difficult debugging	Moderate

Q3) Explain and write Syntax with example of HiveQL DDL, & DML commands for querying

DDL:

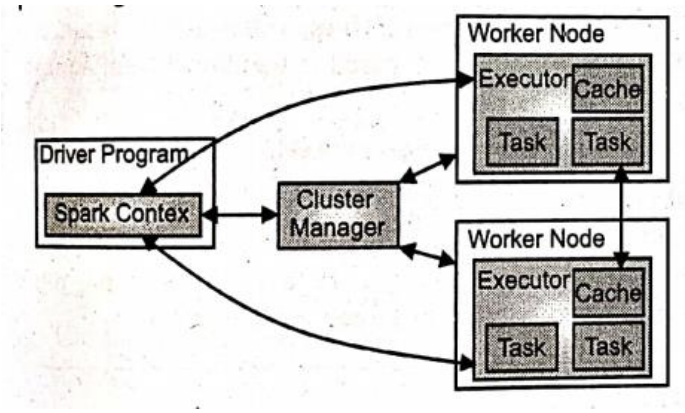
Command	Purpose	Syntax Example	Output / Use
CREATE DATABASE	Creates a new database	CREATE DATABASE student_db;	New database created
SHOW DATABASES	Lists all databases	SHOW DATABASES;	Displays list of databases
USE	Selects a database	USE student_db;	Switches to that database
CREATE TABLE	Creates a table	CREATE TABLE students (id INT, name STRING);	New table created
DESCRIBE TABLE	Shows schema details	DESCRIBE students;	Displays columns and types
ALTER TABLE	Modifies table structure	ALTER TABLE students ADD COLUMNS (marks INT);	Adds/renames columns
DROP TABLE	Deletes a table	DROP TABLE students;	Table removed from database
SHOW TABLES	Lists tables	SHOW TABLES;	Displays all tables in DB

DML:

Command	Purpose	Syntax Example	Explanation
LOAD DATA	Loads data from local or HDFS into a table	LOAD DATA LOCAL INPATH '/home/data/student.csv' INTO TABLE students;	Moves file into Hive table directory
INSERT INTO	Adds new records to an existing table	INSERT INTO TABLE students VALUES (1, 'Ravi', 21, 85);	Inserts or appends data

Command	Purpose	Syntax Example	Use / Explanation
INSERT OVERWRITE	Replaces all existing data with new data	INSERT OVERWRITE TABLE students SELECT * FROM temp_students;	Refreshes table content
EXPORT TABLE	Exports data and metadata to HDFS directory	EXPORT TABLE students TO '/user/hive/export/student_data';	Used for backup or migration
IMPORT TABLE	Imports exported data and metadata	IMPORT TABLE students FROM '/user/hive/export/student_data';	Recreates table from backup

Q4 With a neat diagram describe Spark architecture & Discuss how drivers & Executor co-ordinate distribute computation.



1. **Spark Driver :** The brain of the Spark application, responsible for converting user code into tasks, scheduling these tasks, and coordinating distributed execution. It manages the overall lifecycle of the application.
2. **Spark Executors :** Worker processes launched on cluster nodes that execute tasks assigned by the driver. Executors handle data processing and store intermediate results in memory or disk.

- **Cluster Manager:** Manages resources and handles scheduling across the cluster. Examples include YARN, Mesos, or Spark's standalone cluster manager.
- **Resilient Distributed Dataset (RDD) :** The fundamental data structure in Spark represents an immutable distributed collection of objects, enabling fault-tolerant data processing.

This architecture ensures efficient distributed processing and fault tolerance.

How Driver and Executors Coordinate Distributed Computation

1. The **Driver Program** starts the Spark application.
2. The driver requests resources from the **Cluster Manager**.
3. The cluster manager launches **executors on worker nodes**.
4. The driver divides the job into **multiple tasks**.
5. These tasks are sent to **executors** for parallel processing.
6. Executors process the data and return results to the **driver**.
7. The final result is stored in storage systems such as **HDFS, databases, or files**.

Q5) Create an end-to-end ETL pipeline using Spark & explain each step in detail with reference to the Sample Pyspark code.

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import *

# Step 1: Create Spark Session
spark = SparkSession.builder.appName("SimpleETL").getOrCreate()

# Step 2: Extract
df = spark.read.csv("sales.csv", header=True, inferSchema=True)

# Step 3: Transform
df_clean = df.dropna()
df_filtered = df_clean.filter(df_clean["Amount"] > 1000)
df_final = df_filtered.withColumn("Month", month("Date"))

# Step 4: Load
df_final.write.csv("output/processed_sales", header=True)
```

Step-by-Step Explanation

1. Create Spark Session

```
spark = SparkSession.builder.appName("SimpleETL").getOrCreate()
```

- Initializes the Spark application.
- Creates the **SparkSession**, which is the entry point for using Spark SQL and DataFrames.

---

2. Extract

```
df = spark.read.csv("sales.csv", header=True, inferSchema=True)
```

- Reads the **sales.csv** file.
- `header=True` uses the first row as column names.
- `inferSchema=True` automatically detects data types.
- Data is loaded into a **DataFrame**.

---

3. Transform

```
df_clean = df.dropna()
```

- Removes rows containing **null values**.

```
df_filtered = df_clean.filter(df_clean["Amount"] > 1000)
```

- Keeps only records where **Amount > 1000**.

```
df_final = df_filtered.withColumn("Month", month("Date"))
```

- Creates a new column **Month** extracted from the **Date** column.

---

4. Load

df_final.write.csv("output/processed_sales", header=True)

- Saves the processed data into the **output folder** as CSV files.
- This completes the ETL pipeline.

Summary of Pipeline

1. **Extract** – Read data from CSV.
2. **Transform** – Clean data, filter records, create new column.
3. **Load** – Store processed data into output files.

16) Explain how User Defined Functions' (UDF) are used in Pig for data transformation & normalization

User Defined Functions (UDFs) in Apache Pig allow users to extend the functionality of Pig Latin by writing custom code (typically in Java, Python, or JavaScript) to perform specialized data transformations and normalization tasks that are not supported by built-in Pig functions.

1. Use of UDFs for Data Transformation

Data transformation means converting data from one format or structure into another.

How UDFs help:

- Apply custom calculations on data fields
- Convert data types or formats
- Clean or modify raw data values
- Perform complex logic during data processing

2. Use of UDFs for Data Normalization

Data normalization means organizing and standardizing data into a consistent format.

How UDFs help:

- Standardize text (lowercase/uppercase)
- Remove extra spaces or special characters
- Convert different date formats into one format
- Scale numeric values into a fixed range

(Q7) Explain Data Types & File format of Hive
→ File format – Text file, Sequence file, ORC file, Parquet file, AVRO.
→ Data types :- MAP, STRUCT, TINY INT, SMALL INT, FLOAT, BOOLEAN,

File Formats	Text File	Plain text, one record per line	Easy to read/debug, default format
File Formats	Sequence File	Binary key-value format	Faster read/write, supports compression
File Formats	ORC	Optimized Row Columnar storage	Best for analytical queries
File Formats	Parquet	Columnar storage format	Efficient compression, supports nested data
File Formats	Avro	Row-based with schema support	Ideal for data exchange and ETL pipelines

Primitive Data Types	DECIMAL	Exact numeric values with defined precision	Financial calculations

Complex Data Types	MAP	Key-value pair collection (For example,MAP< string, int>)	Store relationships
Complex Data Types	STRUCT	Group of fields of different types	Record-like data structure

Types		string of characters	storage
Primitive Data Types	STRING	Fixed or	Character
Primitive	CHAR/VAR		

Primitive Data Types	TINYINT	1-byte integer (–128 to 127)	Basic integer data
Primitive Data Types	SMALLINT	2-byte integer (–32,768 to 32,767)	Small range integers

Types			Mathematical computations
Primitive Data Types	FLOAT	4-byte single precision float	
Primitive Data Types	DOUBLE	8-byte double precision float	High-precision calculations
Primitive Data Types	BOOLEAN	Represents TRUE/FALSE values	Logical operations
Primitive Data Types	STRING	Variable-length string of characters	Text data storage

(Q8) Explain HIVE Architecture with its Limitation & Application.

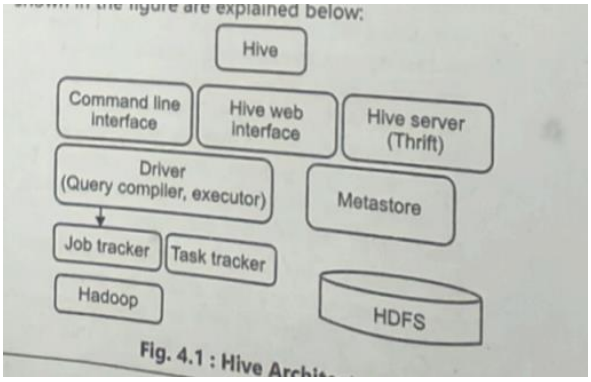


Fig. 4.1 : Hive Archi...

1. User Interface Layer

- Provides interfaces for users to interact with Hive.
- Users submit queries using:
  - **CLI (Command Line Interface)**
  - **JDBC/ODBC**
  - **Web UI or applications**
- Queries are written in **HiveQL**.

2. Driver

- Manages the **lifecycle of a Hive query**.
- Receives the query from the interface.
- Creates sessions and controls execution.
- Sends the query to the compiler.

3. Compiler

- Converts the HiveQL query into an **execution plan**.
- Performs:
  - Syntax checking
  - Semantic analysis
  - Query optimization

4. Metastore

- Stores **metadata information** about Hive tables.
- Includes:
  - Table names
  - Column names and types
  - Partition details
  - Data location in HDFS
- Uses **RDBMS databases** like MySQL or Derby.

6. Hadoop Layer

This is the storage and processing layer.

Components:

- **HDFS** – Stores large datasets
- **MapReduce / Tez / Spark** – Processes the data
- Returns results back to Hive.

4.1.3 Limitations of Hive

1. **Not Suitable for Real-Time Processing:** Hive queries take time since they are converted into MapReduce jobs.
2. **No Row-Level Updates or Deletes:** Hive works best for read-heavy workloads, not frequent data changes.
3. **High Query Latency:** Response time is slower compared to traditional RDBMS.
4. **Limited Transaction Support:** ACID operations are partially supported.
5. **Dependency on Hadoop:** Hive cannot function without the Hadoop ecosystem.

4.1.4 Applications of Hive

1. Data Warehousing and Reporting.
2. Log and Clickstream Analysis.
3. ETL (Extract, Transform, Load) Operations.
4. Business Intelligence and Data Analytics.
5. Large-scale Batch Data Processing.
6. Analysing structured and semi-structured data in HDFS.

9) Explain MongoDB's data model including collections, documents, BSON, indexing, & dynamic schema with detail example.

MongoDB Data Model

MongoDB is a **NoSQL document-oriented database** that stores data in flexible **JSON-like documents** instead of tables and rows.

---

1. Collections

- A **collection** is a group of MongoDB documents.
- It is similar to a **table in relational databases**.
- A collection does not enforce a strict schema.

Example

Collection: students

Documents stored inside the collection.

---

2. Documents

- A **document** is the basic unit of data in MongoDB.

- It is stored in **key–value format** similar to JSON.
- Documents in the same collection can have different fields.

Example

```
{
  "name": "Rahul",
  "age": 20,
  "course": "AI & ML"
}
```

Another document in the same collection:

```
{
  "name": "Asha",
  "age": 21,
  "city": "Mumbai"
}
```

---

3. BSON (Binary JSON)

- MongoDB stores documents in **BSON format**.
- BSON is a **binary representation of JSON-like documents**.
- It supports additional data types like **date, binary, and objectId**.

Example JSON

```
{
  "name": "Amit",
  "age": 22
}
```

In MongoDB it is internally stored as **BSON** for faster processing and storage.

---

4. Indexing

- **Indexes improve the speed of queries**.
- They work similarly to indexes in relational databases.
- Without indexes, MongoDB scans the entire collection.

Example

Create index on name field:

```
db.students.createIndex({name:1})
```

1 → ascending order index.

Benefits:

- Faster search
- Faster sorting

- Efficient query execution

---

5. Dynamic Schema

- MongoDB collections have a **flexible schema**.
- Documents in the same collection **do not need the same structure**.

Example

Document 1:

```
{
  "name": "Riya",
  "age": 19
}
```

Document 2:

```
{
  "name": "Arjun",
  "age": 20,
  "skills": ["Python","ML"]
}
```

Both documents exist in the **same collection even with different fields**.

10) Analyze the BASE VS ACID properties in context of NOSQL scalability & performance.

1. ACID Properties (ACID = SQL/Traditional)

ACID prioritizes **Consistency** over Availability in the CAP theorem.

- **A**tomicity: All-or-nothing transactions.
- **C**onsistency: Data follows strict rules, and all nodes see the same data simultaneously.
- **I**solation: Concurrent transactions do not affect each other.
- **D**urability: Committed data is permanently saved.

Performance & Scalability Impact:

- **Performance (Slow):** Requires locking records during transactions. When a transaction spans multiple nodes (shards), performance decreases due to coordination overhead.
- **Scalability (Vertical):** Primarily scales by increasing machine power (vertical). Horizontal scaling is harder because maintaining "immediate consistency" across servers requires intense communication.

- **Ideal Use Case:** Financial transactions (banking), medical records, or inventory systems where data integrity is absolute.

Amazon Web Services (AWS) +4

---

2. BASE Properties (BASE = NoSQL/Modern)

BASE prioritizes **Availability** and **Partition Tolerance** over Consistency (AP system).

- **B**asically **A**vailable: The system ensures availability by replicating data across multiple nodes.
- **S**oft state: The system state may change over time (due to replication) even without input.
- **E**ventual consistency: All nodes will *eventually* reflect the latest state, but reads might temporarily return stale data.

Performance & Scalability Impact:

- **Performance (Fast):** Offers higher throughput and lower latency. Without strict locking, data can be written to multiple nodes simultaneously.
- **Scalability (Horizontal):** Designed to scale out by adding more commodity servers to a cluster (sharding).
- **Ideal Use Case:** Social media feeds (likes/comments), real-time analytics, user carts, and IoT applications.